腾讯Omega：下一代“AI BI”的答案？

原创 腾讯程序员 腾讯技术工程 2026年8月3日 03:48 广东 375人



作者：jiaxingjin

上传一个 Excel，敲下一句话，等上几十秒，一张带着 KPI、趋势图和排行榜的页面就出现在屏幕上。颜色鲜艳，动画流畅，截图发到群里，很容易得到一句“这个厉害”。

但如果你真打算用它，问题往往从第二天开始。

时间范围从“本月”换成“上月”，数字还停在原地；点一下地区筛选，三个图表各算各的；数据表多了一个字段，整张页面直接空白；想把柱状图换成折线图，AI 又从头生成一遍，把你已经确认的布局也顺手改了。

这时候你会发现，昨天看到的不是一个数据产品，只是一张由 AI 画出来的、恰好很像数据产品的图。

2024 年，我们在 DataTalk 里做过 ChatBI：用户在侧边栏里提问，AI 生成 SQL，再把查询结果画成图。方向听上去没错，实际用下来却很别扭。SQL 经常要人收拾，图表质量不稳定，更关键的是，AI 和原来的 BI 系统始终像两套东西——它能回答一个问题，却接不住后面的分析工作。

后来，我们把它推翻了。

不是给聊天框换个模型，也不是把按钮重新排一遍，而是重新想了一次：如果今天从零设计一个数据分析产品，AI 到底应该站在哪里？

聊天框不是 BI 的下一代

传统 BI 的工作方式很明确：找数据、选字段、拖组件、配筛选器、调样式。它很强，也很专业，但前提是用户已经知道自己要分析什么，还得知道怎么把问题翻译成一组操作。

ChatBI 看似跨过了这道门槛。用户不用拖拽了，直接问：“上个月各品类销售怎么样？”AI 把自然语言翻译成 SQL，查完返回一张图。

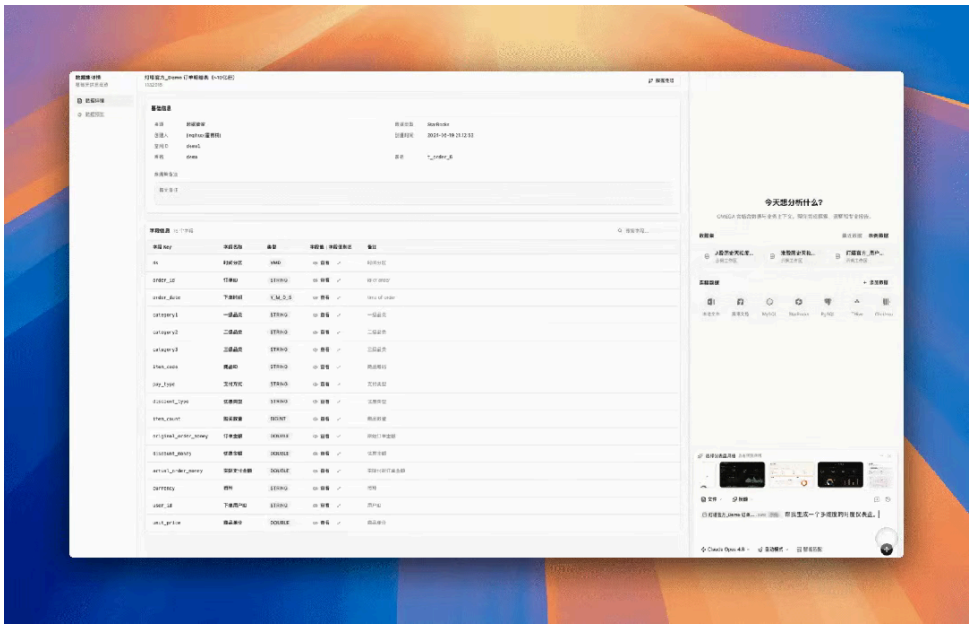
可分析很少在第一张图结束。

你会继续追问：为什么华南区掉得这么厉害？是订单少了，还是客单价下来了？大客户和中小客户的趋势一样吗？把异常最明显的三个城市单独拿出来看看。最后，你可能还要把结果整理

成一个页面，发给团队，并在下周一数据更新时重新看一次。

如果 AI 只能完成“问题到 SQL”这一步，它只是缩短了一次查询，并没有接管一段完整的分析工作。

Omega 想做的是后者。



你说一句“帮我分析上季度各品类的销售情况”，它先理解数据源和字段，再规划这一轮分析应该看哪些指标：总销售额、环比、趋势、品类结构、地区差异。随后自己查询数据、选择图表、组织页面，生成一份完整的 Dashboard。

你觉得第二张图不直观，说“换成柱状图”，它只改第二张图；你想补一个地区筛选器，它把控件加上，并让相关图表一起联动；你觉得配色太重，直接让它调浅一点，也可以打开可视化编辑器自己拖。

页面保存后可以分享，数据会继续刷新，下一次还能沿着这次的结果继续分析。

Omega 的核心开发长期只有 3—4 个人。没有泾渭分明的产品、设计和测试角色，很多决定都是几个人一边写代码，一边争论产品应该长成什么样。2026 年 1 月中旬开工，前两个月把主链路跑通。

但回头看，两个月做出来的只是一个“能演示”的 Omega。后面几个月，我们做的其实是另一件事：让它离开演示环境之后，仍然像个产品。

一块看板，什么时候才算“活着”

AI 生成页面并不难。今天的模型写 HTML、CSS 和 JavaScript 已经相当熟练，配合 ECharts 或 D3，做出一张视觉上不错的看板，门槛比两年前低了很多。

难的是让这张页面和真实数据之间，始终连着一根可靠的管子。

很多 AI 看板的实现，是把查询结果直接写进页面：

```
const data = [
  { month: '1月', sales: 1280 },
  { month: '2月', sales: 1530 }
]
```

这段代码没有错，甚至很适合做 demo。但它生成之后，数据就凝固了。换时间、加筛选、刷新数据，都得重新找 AI 生成一次。页面看着是活的，里面的数据其实已经死了。

Omega 生成页面时，除了 HTML，还会生成一份结构化的数据契约。内部叫 QueryRegistry：每张图需要哪条查询、接收哪些参数、查询结果怎样绑定到图表、它和哪些筛选器联动，都在这里声明清楚。真正的执行交给 DTBridge 运行时。

这有点像把职责拆开：AI 负责表达“我要什么”，系统负责保证“它怎么稳定拿到”。

用户切换日期时，DTBridge 会根据契约刷新受影响的图表。筛选器本身也是声明式的，AI 不需要为每个控件手写 `addEventListener`、拼 WHERE 条件和处理空值。

我们为什么坚持不让模型包办这些代码？因为这类代码单看每一段都不复杂，组合起来却非常容易静默出错。

我们碰到过一个很典型的问题：SQL 明明返回了数据，图表却是空的。数据库没报错，浏览器控制台也没有异常。最后排查发现，页面回调读取的是 `总上报量`，SQL 输出的列名却是 `total_pv`。

两边分别看都合理，合在一起就是拿不到值。

后来，我们增加了字段绑定检查：从页面真实读取的字段反向校验 SQL 输出别名。类似的校验还有很多——时间字段到底是秒还是毫秒、筛选器的值能不能对应查询参数、图表声明里的查询 ID 是否真实存在。它们都不性感，却决定了一块看板是“偶尔成功”，还是“多数时候可以交付”。

数据真正流动起来之后，安全和生成体验也得由系统接管。页面不会拿到数据库账号和密钥，查询与权限校验留在服务端；后端按数据源能力执行 AST 只读校验或轻量只读检查，并叠加查询服务的权限控制与统一超时。页面运行在受限制的 iframe 中。看板流式生成时，我们只更新已经安全到达的样式和正文，完成后再统一加载脚本，避免页面反复白屏。用户看到的只是页面平滑地长出来，背后的边界不能靠模型临场发挥。

AI 怎么找对指标、联动筛选，又怎样守住数据边界

如果你做过 BI，看到这里大概会有几个本能的追问：模型凭什么知道“销售额”对应哪一列、哪一种聚合？企业已经维护的指标口径和语义模型怎么接进来？一个筛选器为什么能同时驱动五张图？一张由 AI 写出来的 HTML，又凭什么被允许查询生产数据？

这些问题不能靠一句“我们用了更强的模型”回答。

Omega 没有绕过 BI 的底座。DataTalk 已有的数据源接入、元数据、语义模型、查询引擎和权限体系仍然在下面。Agent 负责理解问题、组织分析和生成界面，但涉及数据含义与执行边界时，它必须回到系统能够验证的证据上。

从一句业务问题到一块能运行的页面，实际走的是这样一条链路：



先找证据，再谈“理解指标”

假设用户说：“看一下上个月各地区的销售额。”真正困难的不是写出 `SUM`，而是确定“销售额”到底是什么：含税还是不含税，是否扣除退款，按支付时间还是下单时间统计。企业里两个名字相同的指标，口径完全可能不一样。

Omega 找指标时有一条很重要的原则：离用户当前工作现场越近的证据，优先级越高。

如果用户正在修改一张已有图表，最可信的不是 LLM 对“销售额”的常识，而是这张图已经执行过的真实 SQL。系统会把当前图表的 `datasetId`、查询说明、数据库引擎和 SQL 一起交给 Agent。SQL 里如果写的是 `SUM(pay_amount - refund_amount)`，或者 `COUNT(DISTINCT buyer_id)`，这就是当前页面已经采用的业务定义；后续增加地区维度、改图表类型时，应当复用这个定义，而不是重新发明一遍。

如果是一次新分析，用户在消息里明确选择的数据集优先于历史页面和系统搜索结果。生成前，Agent 必须先调用 Schema 工具，拿到真实的物理字段、展示名、类型、字段说明、分区信息、数据源引擎和连接路由。这个动作不只写在 prompt 里，外层还有 Schema Guard：用户已经指定了数据，模型却没有读 Schema 就开始生成页面，流程会被拦住。

用户没有明确指定数据时，系统才会在个人空间、团队空间和可访问的数据目录中检索。检索不是把所有表名一股脑塞给模型，而是同时看名称、描述、物理表名、关键词覆盖、数据归属和更新时间，合并去重后给出候选。它解决的是“从几千个数据集中缩小范围”，并不假装自己已经理解了缺失的业务口径。

还有一类证据来自业务知识。用户可以引用指标说明、分析方法或业务文档，Agent 会把相关片段带进本轮上下文。于是“有效成交额要扣除全额退款”“新客按首次支付日期认定”不再只是聊天中的一句提示，而会和字段元数据、已有查询一起参与规划。

这几层信息构成的，其实是一条指标证据链：**已有查询口径 → 用户指定的数据 → 语义模型和字段元数据 → 被明确引用的业务知识 → 检索候选**。证据足够，系统才有条件自动生成；如果上游既没有指标定义，用户也没有说明，仅凭一个相似列名，系统就不具备自动判定准确口径的条件。更诚实的做法是暴露候选或继续确认。LLM 可以降低使用数据的门槛，不能顺手替企业补完从未做过的数据治理。

有语义模型时，不让模型猜物理 JOIN

很多系统所谓“支持语义”，其实只是把字段注释拼进 prompt。我们接入的语义模型路径更具体。

普通数据集走物理 Schema：Agent 看到真实表和字段，按对应数据库方言生成 SQL。语义模型则走另一条链路。系统先取得模型中的逻辑字段和角色，把它们分类为维度 `DIM`、指标 `FACT` 和时间维度 `TIME_DIM`，同时保留展示名、描述、类型以及模型标识。

例如用户问“各渠道近三个月的付费转化率”，Agent 使用的是“渠道”“付费转化率”“统计日期”这些经过治理的逻辑对象，先生成基于 `MODEL('<modelCode>')` 的 SemanticQL；随后由语义查询服务把它转换成真正执行的物理 SQL。底层事实表怎样关联、指标公式怎样展开、不同存储引擎怎样适配，不再由模型临场猜测。

转换失败时，系统也不会把一段有问题的 SQL 直接扔给用户。转换错误、上一版 SemanticQL 和可用字段列表会一起反馈给 Agent；遇到 `Unknown identifier`，它只能从已经拿到的真实字段中重新选择，并在限定次数内修复。这里的重试是有依据的纠错，不是让模型对着同一个答案反复碰运气。

当然，语义模型只能放大已有治理的价值，不能创造不存在的口径。上游定义了统一指标、逻辑字段和关系，Omega 才能稳定消费；上游只有一堆命名随意的宽表，系统可以通过 Schema 校验拦住不存在的字段，却不能保证自动猜中每家公司独有的业务含义。这是我们愿意保留的一条边界。

联动筛选，本质上是一张查询依赖图

指标找到之后，页面还不能把结果直接写死。Omega 会在 HTML 之外生成一份结构化数据契约 QueryRegistry。下面是一条经过精简的查询声明：

```
{
  "queryId": "q_sales_trend",
  "datasetId": "sales_dataset",
  "sqlTemplate": "SELECT ds, SUM(gmv) AS sales FROM orders WHERE ds BETWEEN {{startDate}}
  "params": [
    { "name": "startDate", "type": "date", "filterType": "dateRange", "pairWith": "endDat
    { "name": "endDate", "type": "date", "valueType": "str_YYYY-MM-DD" }
  ],
  "fieldBindings": [
    { "alias": "ds", "role": "dimension", "targetType": "chart-axis" },
    { "alias": "sales", "role": "metric", "targetType": "chart-series" }
```

```
  ]
}
```

`queryId` 是页面和查询之间的稳定引用；`datasetId` 把请求路由到已经登记的数据集；`params` 保存参数类型、日期精度、默认值、成对关系、级联来源和选项查询；`fieldBindings` 则明确 SQL 输出中的哪一列是维度、哪一列是指标，避免图表读取 `销售额`、SQL 却返回 `total_gmv` 这类静默失败。

页面不负责临时拼接一套联动逻辑。DTBridge 加载时会扫描每条 SQL 模板中的占位符，构建一张 `参数名 → queryId[]` 的依赖表。假如 `region` 同时出现在销售趋势、品类结构、客户排行和地图四条查询中，地区筛选器更新后，系统可以直接计算出受影响的查询集合，只刷新这四张图；和 `region` 无关的库存卡片不动。

真实业务里的时间字段还经常不是一种格式。同一个日期筛选值，在事实字段里可能要变成 `2026-07-22 00:00:00`，在分区字段里却要变成 `20260722`。QueryRegistry 可以通过 `valueType` 记录格式，再用 `aliasOf` 让两个 SQL 参数共享同一个筛选器的标准值。格式转换在运行时统一完成，模型不需要在每张图里各写一份日期处理代码。

连续拖动或快速切换筛选器时，多次变化会先被防抖合并；同一查询未完成的旧请求会被取消；不同图卡的查询并发刷新，并分别维护 loading。级联筛选也沿着同一张依赖图工作：省份变化后，城市选项查询引用了省份参数，运行时就会先更新城市候选，再刷新依赖城市的图表。

参数注入会根据类型处理日期、秒和毫秒，对字符串字面量做转义，去掉分号和注释片段；缺失的必填值会留下未解析标记，由调用方阻止执行，“全部”则通过条件块或空集合语义处理。但我们并不把这层转义包装成“绝对防注入”——不同引擎并非都能使用同一种预编译参数机制，真正的安全还需要服务端的查询防线。



安全不是一个 iframe 属性

“数据不泄露”很容易写成一句漂亮但不负责任的话。只要结果已经展示给有权限的用户，这个用户当然能够看到它。系统真正要保证的是：无权限的人拿不到；每次查询都先通过当前访问者及分享关系的授权，再绑定服务端解析出的有效查询身份；生成页面拿不到源凭证，查询也不能越过允许的边界。

Omega 的第一道边界在身份和授权。查询身份由服务端从当前登录态解析，不接受页面自己声明一个 `userId`。普通访问先校验文件和数据集权限；分享访问先校验分享关系，再解析这次查询应使用的有效身份。数据集 Schema、查询执行和文件打开分别做权限判断，不能因为 Agent 曾经拿到一个数据集 ID，就默认后续所有请求都合法。

第二道边界在查询。对能够完整解析的 SQL 方言，后端使用 AST 检查保证单条、只读 `SELECT`，拒绝多语句和危险函数，提取包括子查询、CTE 在内的真实表引用，并统一加上行数上限。对 ClickHouse 这类无法完全复用同一解析器的引擎，则采用较轻的只读检查，同时依赖驱动的 `readonly` 模式、查询超时和结果上限。安全策略要尊重数据源差异，不能假装一条正则或者一个通用解析器能覆盖所有数据库。

第三道边界才是页面运行环境。数据库账号、密码和访问密钥不会进入生成的 HTML，页面只能通过受控查询代理请求数据；iframe sandbox 和 CSP 用来限制生成代码的能力，独立预览还会收紧网络连接范围。但我们不把 iframe 当成数据防泄漏系统：页面仍可能持有查询契约，也一定会拿到获准展示的查询结果。对高度敏感的数据，仍然需要上游行列权限、字段脱敏以及更严格的网络出口策略。

还有一个容易忽略的身份生命周期：持久化时，系统会剥离临时注入的运行时脚本和查看者信息；真正打开或分享时，再注入唯一一份 QueryRegistry、DTBridge SDK 和本次访问的系统参数。数据契约可以保存，运行时身份不能被永久烘焙进快照。

生成完成后，系统还会做结构和契约层的静态检查：参数有没有对应 SQL 占位符、全局筛选器是否漏掉某条查询、选项查询是否存在、图表读取字段能否与 SQL 输出别名对齐。初次写入后再做 SQL 预验证，发现可修复的问题则生成新版本；只改视觉风格时，尽量复用已经验证过的 QueryRegistry，不让一次换色顺手改掉指标口径。

所以，Omega 里的模型输出不是最终产物，更像一份需要经过链接、类型检查、权限检查和运行时调度的源代码。模型负责提高表达能力，确定性的系统负责把它约束成一个可以持续运行的数据产品。

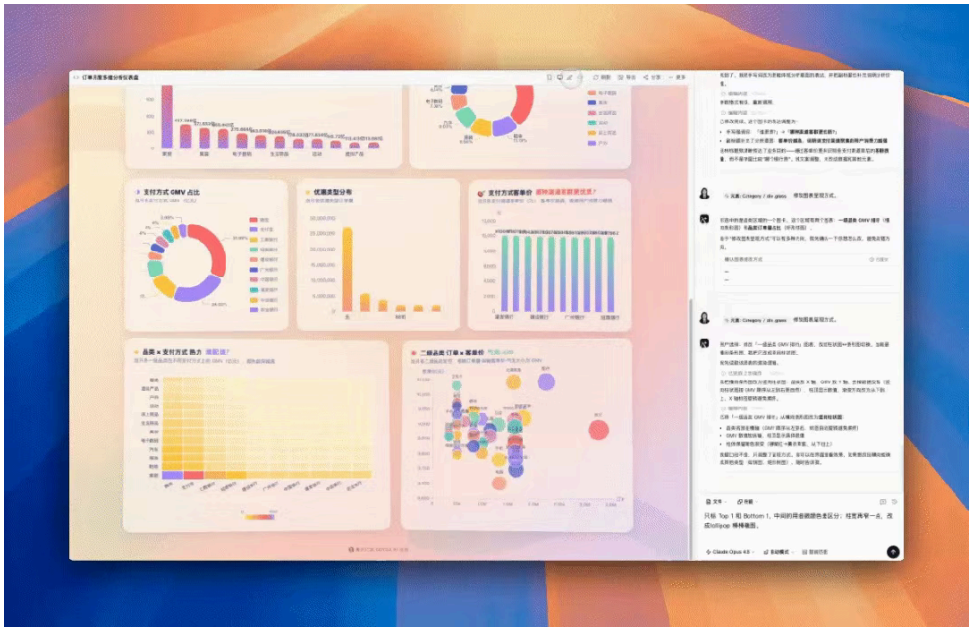
AI 生成完，人还得接着工作

人很少能在第一遍就把需求说完整。看见结果之后，才会知道哪里不对：标题太长，图选错了，某个指标应该拆开，筛选范围还不够。一个真正进入工作流的产品，必须允许人继续接手，而不是逼用户在“接受整页”和“全部重来”之间二选一。

小改动可以直接告诉 AI，“把第二张图换成折线图”，它精准定位目标区域，只改那一块；需要微调布局和样式时，用户可以在 GUI 里拖拽；变化比较大时，Plan Mode 会先列出准备修改的内容，确认后再执行，避免一句模糊的要求把整张页面改得面目全非。

底层之所以能支持这种协作，和我们一直在维护的 A-MD 协议有关。A-MD 是外层文件协议：一个 AMDFile 同时包含 SQL、Python、图表、指标、筛选器和说明文字等 Block，以及不同视图的 Layout。Canvas 不是另一套并列模型，而是 `dashboardLayout.canvas` 下的一种 Dashboard 视图载荷，主要保存 HTML 与 QueryRegistry。A-MD 管文件、版本和可编辑的分析过程，Canvas 负责最终页面的动态查询。

我们不想把 GUI 消灭掉。自然语言适合表达意图，鼠标适合精确调整，代码适合处理复杂逻辑。AI 原生不等于所有交互都必须经过聊天框，而是产品知道什么时候该让 AI 接手，什么时候该把控制权交还给人。



好看不是锦上添花

做数据产品的人容易低估审美，因为数据准确显然比颜色重要。但真实世界里，一份没人愿意打开第二次的分析，很难产生多少影响。管理者看到的是页面，不是背后的 SQL；业务同学愿不愿意把结果发到群里、放进汇报，往往在几秒钟内就决定了。

AI 在这件事上有一种顽固的平均脸。让它自由发挥做十张看板，大概率会得到十张“白底、蓝色柱状图、四个 KPI 卡片排一行”的页面。偶尔换成深色，就是熟悉的紫色渐变。我们试过在 prompt 里多写几句“高级、科技、克制”，效果有改善，但很不稳定。

于是我们给 Omega 做了一套审美基础设施。现在积累了 23 种视觉风格和 18 种可视化引擎，但它们不是用来换皮肤的模板。赛博朋克监控大屏、移动端经营日报、极简财务报告，差的不只是背景色，还有信息密度、字体层级、留白、图表选择和交互方式。

这套系统有一个我们内部常说的设计：薄配置，厚引擎。一份内置风格的源配置只保留几个颜色、一个字体、几个关键词和简短的调性描述，大约 120 字；后端把它展开成一份约 800 字的设计规格，补齐图表色板、文字关系、边框、布局和动效约束。

审美系统最后要解决的不是炫技，而是两个很现实的问题：同样的数据，能不能让人更快看懂；同样的结论，用户愿不愿意截图发给老板。



高性价比模型，也能把复杂看板做好

做出第一个效果不错的页面之后，我们很快碰到另一个诱惑：只要换更前沿的模型，问题是不是就解决了？

前沿模型当然有价值。它在复杂布局、代码生成和一次成型率上通常更好。但数据分析产品不可能只看最好的一次，还得考虑每天大量请求的成本，以及最差情况下系统会做出什么。

我们做过一个内部固定样例：同一份生成并修改一次看板的任务，Claude 4.6 的模型调用费用约 14.2 元，混元 Hy3 约 0.47 元，相差约 30 倍。这个数字只是一组固定任务下的成本样本，不是两种模型质量的通用排名，但它足以让我们认真思考：有没有可能让性价比更高的模型，也稳定完成复杂工作？

真正开始做之后，我们发现有一些模型最麻烦的地方，并不是它会明确告诉你“我失败了”，而是它会交出一份看起来已经完成的结果。

HTML 被截断了，它仍然有标题和前三张图；SQL 查到了数据，页面绑定的是另一个字段名；时间筛选器传的是毫秒，数据库字段存的是秒；模型凭记忆写出一个并不存在的 SDK API；数据里明明只有三张表，它顺手编出第四张，而且名字还很像真的。

这些问题共同的特点是：没有明显的红灯。用户只有打开页面、点击筛选器，甚至等到下周数据变化之后，才发现它不工作。

我们后来不再把稳定性完全押在模型的“自觉”上，而是在模型外面加了一套 Harness。可以把它理解成 Agent 的工作环境：模型拿到什么上下文，可以调用哪些工具，任务怎样拆分，输出经过哪些校验，失败之后怎样反馈和修复。

工具层也没有和某个 Agent 框架绑死。运行时会根据当前 Canvas、文件类型、用户选择和计划阶段，自动预激活高置信度工具；剩下的能力再由模型通过 select_tools 渐进选择，避免几十份工具 Schema 同时进入上下文。业务工具遵循统一的框架无关契约，底层更换 Agent 框架时，主要调整适配层。

现在，一次看板生成不会直接从模型输出跳到用户面前。系统先检查结构是否完整、查询声明是否存在、图表有没有绑定到真实查询；遇到截断，会裁掉不完整片段、补齐必要的闭合结构，再继续生成。字段别名、时间格式、筛选器选项这类确定性问题优先用规则处理，表名则尽量从真实元数据中选择并在执行前验证。

规则能修的，不让模型重写；规则无法判断的语义问题，再交给 Agent 带着错误上下文做一次有目标的修复。

让模型面对自己刚生成的一大段代码，说“请检查并修复所有问题”，看起来聪明，实际上既贵又不可测试，还可能修好一处、改坏另一处。把错误拆成类型之后，系统才能逐个积累确定的解法。写入结果前，Omega 还会重新验证当前用户的权限，不能因为 Agent 拿到了一个文件 ID，就默认它有权覆盖。

一条慢 SQL 暴露出的运行时问题

Omega 上线早期发生过一次很典型的事故。

一位用户发起分析，其中一条 SQL 在计算高基数字段的 `COUNT(DISTINCT)`。查询本身没有死锁，也没有报错，只是执行期间不会返回中间数据。后来多次复现，同类查询通常需要 110—130 秒。

当时系统里有一个 120 秒的空闲看门狗：这么久收不到数据，就认为连接卡死并把它终止。结果，一个正在正常工作的慢查询，被当成僵尸杀掉了。

用户端没有得到明确报错，回答只是写到一半突然停住。更糟的是，前端连接虽然断了，停止信号却没有传到最底层，那条已经没人需要的 SQL 仍在后端运行，直到 30 分钟后才超时。

这次事故让我们意识到，Agent 运行时不能只靠一堆散落的 timeout。它至少要回答四个问题：每一步最多执行多久？用户停止后，资源能不能真的释放？无论怎样结束，前端能不能知道原因？一次局部失败，模型有没有机会换条路继续？

后来我们把它们收成四个运行时契约：有界、可取消、可观测、可自纠。

现在，SQL 这类叶子工具的 deadline 由工具自己声明，再由统一适配层执行，而不是散落在各段业务代码里；用户点击停止后，取消信号会一直传到真正执行查询的地方。每轮运行无论正常结束、超时还是被取消，都必须返回且只能返回一个类型化的 `finish{reason, detail}`。即使异常终止，前端也会保留已经生成的内容，而不是把整段回答抹掉。

持续流式输出的子 Agent 则不能照搬叶子工具的超时方式。运行时不会靠猜测判断，而是明确对 `delegate_agent_*` 和标记为 `backend_agent` 的工具豁免 deadline-race，运维配置也不能把它重新武装；这类任务改用整轮生命周期上限、活性检测和流式心跳管理。

失败也不一定意味着整个任务失败。工具超时后，模型会拿到明确原因，再决定缩小查询或换一种策略。我们也给重复调用加了护栏——线上曾经出现过模型对同一个工具连续发起 32 次完全相同的请求。现在系统会先提醒它换路，继续重复到阈值后直接熔断。

这些细节很少出现在产品演示里。但一个 Agent 能不能上线，往往不取决于它顺利时有多聪明，而取决于它超时、截断、重复、取消时，有没有一个体面的收场。

对话结束之后，Agent 才开始变得有价值

有段时间，我们判断一张看板是否完成，标准是“生成成功并保存”。后来越用越觉得不对。

业务真正需要的，通常不是今天看一次数据，而是以后别忘了继续看。

现在，Omega 可以在任意图卡上设置监控规则。比如“转化率低于 5%，并且订单量高于 1000”才告警，也可以跨多张卡片组合条件。后台按计划重新执行查询，满足条件后通过企业微信或邮件通知。

看板也可以按周或按月定时推送。到时间后，系统会绕过旧缓存，用最新数据重新截图。某一部分查询失败，不会让整次推送直接消失：可以展示的内容仍会尽量发送，失败详情则保留在推送记录中。对数据产品来说，有记录地降级，通常比悄悄什么都不给更有用。

每次编辑都会留下版本，用户可以看 diff、回到之前的结果。复制或发布时，版本历史会跟随产物；权限不会被原样照搬到另一个空间，而是在目标空间重新授权。访问记录、团队协作和企业微信机器人，则让这份分析逐渐离开个人工作台，进入真实的协作环境。

Skills 和 MCP 也在把这些能力接到 Omega 之外。用户最终不关心协议名，他关心的是：能不能在企业微信里问同一个问题，能不能让另一个 Agent 调用这份分析，能不能在数据异常时主动收到消息，而不是每次都重新打开一个聊天框。

写在最后

我们现在仍然不敢说 Omega 已经是“下一代 BI”的答案。真实数据比演示数据脏得多，企业里的权限和流程也比一句 prompt 复杂得多，很多体验仍在快速变化。

在 AI 时代，会生成页面会越来越便宜。真正稀缺的，是生成之后，那张页面第二天还能不能用；那次分析下一周还能不能继续；那个 Agent 出错的时候，仍然值不值得信任。

目前我们已经发布了内测版本「马尔摩斯/marmos」（marmos.qq.com），欢迎联系我们。





腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

688篇原创内容

公众号



阅读 1.8万

留言

写留言

还没有留言，来写首评